

JEGYZŐKÖNYV

Webes adatkezelő környezetek

Féléves feladat

MyList

Készítette: Lőrinc Ákos

Neptunkód: XVG56E

Dátum: 2025. december

Miskolc, 2025

Tartalomjegyzék

Bevezetés	3
1. Első feladat	3
1.1 Az adatbázis ER modell tervezése	3
1.2 Az adatbázis konvertálása XDM modellre	4
1.3 Az XDM modell alapján XML dokumentum készítése	4
1.4 Az XML dokumentum alapján XMLSchema készítése	6
2. Második feladat	7
2.1 Adatolvasás	7
2.2 Adat-lekérdezés	9
2.3 Adatmódosítás	9

Bevezetés

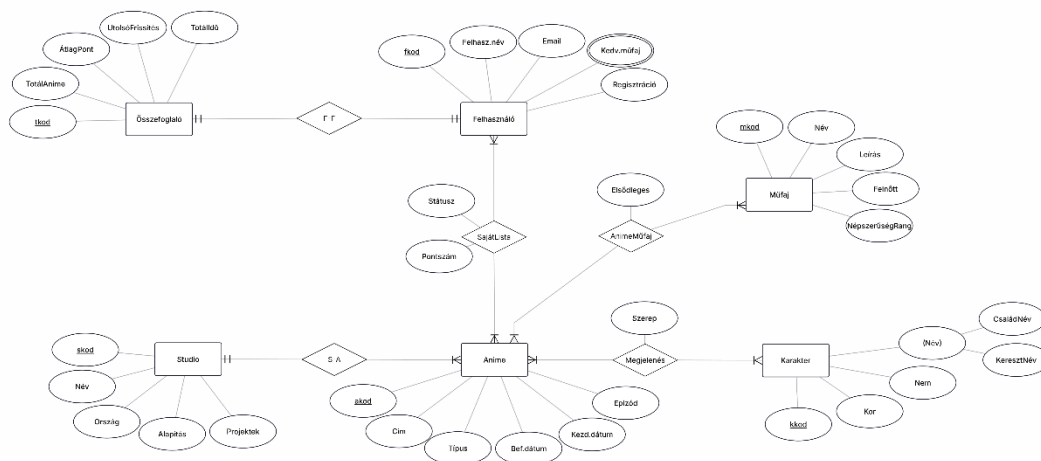
A beadandó feladat célja egy összetett, XML alapú adatkezelő rendszer megvalósítása, amely működésében a MyAnimeList online adatbázis egyszerűsített modelljét követi. A rendszer feladata különböző anime-adatok, felhasználók, stúdiók, karakterek, műfajok és ezek kapcsolatai tárolása strukturált XML formában, majd ezeknek az adatoknak a feldolgozása DOM (Document Object Model) technológia segítségével. Az XML dokumentum felépítéséhez elkészítettem a rendszer ER-modelljét, amely legalább öt egyedet tartalmaz (felhasználó, összefoglaló, stúdió, anime, karakter, műfaj), továbbá többféle kapcsolatot is (1:1, 1:N, M:N). Külön figyelmet fordítottam azokra a kapcsolóelemekre, amelyek N:M kapcsolatot valósítanak meg, és saját attribútumokkal rendelkeznek – ilyen például az „animeműfaj” vagy a „sajátlista”, amelyek az animék és műfajok, illetve a felhasználók és animék közötti összeköttetést tartalmazzák.

A projektben több XML-technológiai feladatot kellett megvalósítani. Először a teljes XML dokumentumot XSD séma segítségével formálisan leírni, amely meghatározza az elemek sorrendjét, kötelező attribútumait, adattípusait és az egyes kapcsolatok érvényességét. Ezt követte az adatolvasás: DOM parser segítségével a program beolvassa az XML állományt, és a teljes tartalmat blokkos formában a konzolra, illetve fájlba írja. A második nagyobb részfeladat az adatlekérdezés volt, ahol XPath használata nélkül, tisztán DOM műveletekkel kellett különböző információkat kinyerni (például egy anime karakterei, egy felhasználó listáján lévő animék, vagy egy adott műfajú animékre történő szűrés). Végül az adatmódosítás során a DOM fa szerkezetét kellett dinamikusan módosítani: attribútumok átírása, elemek bővítése, új csomópontok beszúrása és az eredmény visszaírása egy új XML fájlba.

1. Első feladat

1.1 Az adatbázis ER modell tervezése

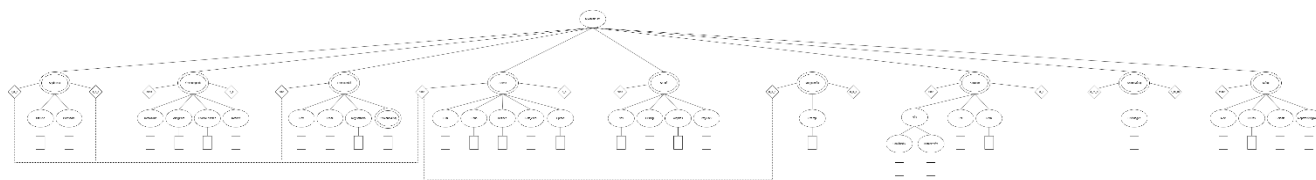
Az XML adatkezelő rendszer megtervezésének első lépése az entitás–kapcsolat (ER) modell elkészítése volt, amely a tárolt adatok szerkezetét, valamint az egyes egyedek közötti összefüggéseket írja le. A feladat követelményeinek megfelelően a modell legalább öt önálló entitást tartalmaz, amelyek a következők: **felhasználó**, **összefoglaló**, **stúdió**, **anime**, **karakter**, **műfaj** és többféle kapcsolat is megjelenik.



1. ábra Kép az ER modell

1.2 Az adatbázis konvertálása XDM modellre

A tervezett ER modell alapján a következő lépés az adatok XDM (XML Data Model) szerinti reprezentációja volt. Az XDM modell leírja, hogyan jelennek meg az entitások, kapcsolatok, attribútumok és hierarchiák XML formában, és hogyan konvertálható a logikai adatmodell strukturált, fa alapú XML-dokumentummá. Az XDM konverzió során minden entitásból XML elem (node), minden attribútumból pedig vagy XML attribútum, vagy gyermekelem jön létre. Az 1:N kapcsolatokat egyszerű referencia-attribútumokkal oldottam meg (pl. az anime a_s attribútuma a stúdió azonosítójára mutat), míg az M:N kapcsolatokhoz önálló XML elemeket hoztam létre (<megjelenes>, <animemufaj>, <sajatlista>), melyek az érintett entitások kulcsaira hivatkoznak, és saját attribútumaikat is tartalmazzák. Ezzel a megoldással az XDM modell nemcsak az adatokat, hanem a köztük lévő kapcsolati szabályokat is képes természetes módon leképezni.



2. ábra Kép az XDM modellről

1.3 Az XDM modell alapján XML dokumentum készítése

Miután az ER modellből előállt az XDM modell, a következő lépés a tényleges XML dokumentum elkészítése volt. Az XDM modell határozta meg, hogy az egyes entitások milyen XML elemek formájában jelennek meg, hogyan ágyazódnak egymásba, illetve milyen attribútumokat vagy gyermekcsomópontokat tartalmaznak. A dokumentum gyökerét a **<myList>** elem adja, amely minden további adatot hierarchikus formában foglal össze. Az egyes egyedekhez külön elemcsoportot hoztam létre: **<felhasznalo>**, **<studio>**, **<anime>**, **<karakter>**, **<mufaj>**, valamint a kapcsolati elemeket reprezentáló **<megjelenes>**, **<animemufaj>** és **<sajatlista>** elemek is ebbe a struktúrába illeszkednek.

A kulcs attribútumok – például a felhasználók fkod, az animék akod vagy a stúdiók skod azonosítója – XML attribútumként kerültek be, mivel ezek egyértelmű azonosítást biztosítanak és több helyről is hivatkozás történik rájuk. A normál attribútumok gyermekelemek formájában jelennek meg: ilyen például a felhasználó neve, e-mail címe, a stúdió országadata vagy az anime epizód száma. A modellben szereplő összetett és többértékű attribútumok is visszaköszönnek az XML szerkezetében: például a karakter neve több gyermekelemből (**<csaladnev>**, **<keresztnev>**) áll, míg a felhasználó kedvenc műfajai ismétlődő **<kedvencmufaj>** elemeként jelennek meg.

```
<!-- SajátLista -->
<sajatlista f_a_a='a1' f_a_f='f1'>
  <statusz>Befejezett</statusz>
  <pontszam>8</pontszam>
</sajatlista>
<sajatlista f_a_a='a1' f_a_f='f2'>
  <statusz>Megtekintés</statusz>
  <pontszam>7</pontszam>
</sajatlista>
```

3. ábra Kódrészlet az XML fájlból

Az alábbi ábrán látható a **<sajatlista>** elem, amely segítségével a **<felhasznalo>** és **<anime>** elemek lettek összekapcsolva.

1.4 Az XML dokumentum alapján XMLSchema készítése

Az elkészült XML dokumentum alapján létrehoztam egy XML Schema (XSD) állományt, amely formálisan meghatározza az XML szerkezetét és az egyes elemek felépítését. A séma célja az volt, hogy az XDM modellben megtervezett struktúrát pontosan leképezze, és biztosítsa az adatok érvényességét. Az egyedekhez (felhasznalo, studio, anime, karakter, mufaj) külön komplex típusokat hoztam létre, melyek meghatározzák az elemhez tartozó gyermekelemeket, adattípusokat és kötelező attribútumokat. Az M:N kapcsolatokat megvalósító elemek (megjelenes, animemufaj, sajátlista) szintén saját típusokat kaptak, a hivatkozások érvényességét pedig xs:key és xs:keyref definíciók biztosítják. Az XSD segítségével az XML dokumentum érvényesíthetővé vált, és jól strukturált alapot biztosít a későbbi DOM feldolgozási feladatokhoz.

```
<xs:complexType name="karakterTipus">
  <xs:sequence>
    <xs:element name="nev" type="karakterNevTipus"/>
    <xs:element name="kor" type="xs:int"/>
    <xs:element name="nem" type="xs:string"/>
  </xs:sequence>
  <xs:attribute name="kkod" type="xs:string" use="required"/>
</xs:complexType>

<xs:complexType name="karakterNevTipus">
  <xs:sequence>
    <xs:element name="csaladnev" type="xs:string"/>
    <xs:element name="keresztnev" type="xs:string"/>
  </xs:sequence>
</xs:complexType>
```

4. ábra Kódrészlet az XSD fájlból

Az alábbi ábrán a <karakter> elemhez elkészített komplex típus látható, ami rendelkezik egy karakterNevTipus nevű komplex típussal.

2.Második feladat

2.1 Adatolvasás

Az XML dokumentum beolvasásához DOM alapú megoldást használtam. A DocumentBuilderFactory segítségével létrehozok egy DocumentBuilder objektumot, amellyel a myAnimeList.xml fájlt parse-olom. Az eredmény egy Document objektum, amely a teljes XML fát reprezentálja. A gyökérelem normalizálása után külön metódusokkal járom be az egyes elem-típusokat (felhasznalo, studio, anime, stb.). Minden ilyen metódus a getElementsByTagName függvény segítségével lekéri az összes megfelelő elemet, majd a kapott Element objektumokból kiolvassa az attribútumokat és a gyermekelemek szövegét. Az adatokat blokkos formában kiírom a konzolra, illetve ugyanilyen struktúrában egy szövegfájlba (myAnimeList_output.txt) is eltárolom egy PrintWriter segítségével. A kódban több kommentet is elhelyeztem, amelyek magyarázzák a főbb lépéseket (DOM fa felépítése, elemlisták bejárása, gyermekelemek kiolvasása).

```
private static void printSajatListak(Document doc, PrintWriter writer) {
    NodeList list = doc.getElementsByTagName("sajatlista");
    writer.println("==== SAJÁT LISTA ====");
    System.out.println("==== SAJÁT LISTA ====");

    for (int i = 0; i < list.getLength(); i++) {
        Element sl = (Element) list.item(i);

        String animeRef = sl.getAttribute("f_a_a");
        String felhRef = sl.getAttribute("f_a_f");
        String statusz = getTagText(sl, "statusz");
        String pontszam = getTagText(sl, "pontszam");

        writer.println("Felhasználó: " + felhRef + " - Anime: " +
animeRef);
        writer.println("  Státusz: " + statusz);
        writer.println("  Pontszám: " + pontszam);
        writer.println();

        System.out.println("Felhasználó: " + felhRef + " - Anime: " +
animeRef);
        System.out.println("  Státusz: " + statusz);
        System.out.println("  Pontszám: " + pontszam);
        System.out.println();
    }
    writer.println();
    System.out.println();
}
```

A `printSajatListak()` metódus feladata a felhasználók és animék közötti M:N kapcsolatot reprezentáló `<sajatlista>` elemek bejárása és kiírása. A függvény DOM segítségével lekéri az összes ilyen elemet, majd minden bejegyzésből kiolvassa az idegen kulcsokat (`f_a_f`, `f_a_a`), a státuszt és a pontszámot. A metódus a kiolvasott adatokat jól áttekinthető, blokkos formában írja ki a konzolra és a kimeneti fájlba. A kódrészlet szemlélteti, hogyan lehet komplex kapcsolati adatokat DOM segítségével feldolgozni és megjeleníteni.

```
private static String getTagText(Element parent, String tagName) {  
    NodeList list = parent.getElementsByTagName(tagName);  
    if (list.getLength() == 0) {  
        return "";  
    }  
    return list.item(index: 0).getTextContent().trim();  
}
```

5. ábra Segédmetódus a gyerekelemek olvasásához

A `getTagText()` segédmetódus egy adott XML elem gyermekelemeinek egyszerű és biztonságos kiolvasását végzi. A függvény a `parent.getElementsByTagName(tagName)` segítségével megkeresi a megfelelő nevű gyerekelemet, majd ha létezik, visszaadja annak szöveges tartalmát, egyébként üres stringet ad vissza. Ez a metódus jelentősen leegyszerűsíti a DOM-ból történő adatkinyerést, hiszen elkerülhető vele a többször ismétlődő null-ellenőrzés és beágyazott node-kezelés.

2.2 Adat-lekérdezés

Az adatlekérdezéseket DOM alapú feldolgozással valósítottam meg, XPath használata nélkül. A DocumentBuilderFactory és DocumentBuilder segítségével beolvasom a myAnimeList.xml fájlt egy Document objektumba. A lekérdezéseket külön metódusokba szerveztem, pl. queryAnimekStudioNevvel, queryKarakterekEgyAnimeben, queryFelhasznaloListaja, queryAnimekMufajAlapjan. Ezek a metódusok az getElementsByTagName és a DOM fa bejárásával szűrik ki a megfelelő elemeket, majd attribútumok és gyermekelemek alapján összekapcsolják az egyes egyedeket (pl. anime – studio, megjelenes – karakter, sajátlista – felhasznalo – anime, animemufaj – mufaj). A lekérdezések eredményét a konzolra írom ki, ember által olvasható, blokkos formában.

```
//Id alapján megkeresi, hogy van-e olyan műfaj
private static String findMufajIdByNev(Document doc, String mufajNev) {
    NodeList list = doc.getElementsByTagName("mufaj");
    for (int i = 0; i < list.getLength(); i++) {
        Element m = (Element) list.item(i);
        String nev = getTagText(m, "nev");
        if (mufajNev.equals(nev)) {
            return m.getAttribute("mkod");
        }
    }
    return null;
}
```

A findMufajIdByNev metódus egy egyszerű DOM bejárást valósít meg: végigmegy az összes <mufaj> elemen, elolvassa a <nev> gyermekelem szövegét, és ha az egyezik a paraméterben kapott névvel, visszaadja az mkod attribútum értékét. Ezzel a módszerrel tudom a szöveges műfajnévből kiszámítani a hivatkozott azonosítót, amit aztán az animemufaj kapcsolóelemekben tudok felhasználni.

2.3 Adatmódosítás

Az adatmódosítást DOM alapú feldolgozással valósítottam meg. A DocumentBuilderFactory és DocumentBuilder segítségével beolvasom a myAnimeList.xml fájlt egy Document objektumba. A módosításokat külön metódusokba szerveztem (modifyFelhasznaloEmail, addKedvencMufajToFelhasznalo, modifySajatlistaPontszam, addNewAnime), hogy jól elkülönüljenek a különböző műveletek. Minden metódus DOM műveleteket használ (getElementsByTagName, attribútumok lekérdezése, setTextContent, appendChild), és a módosítások előtt/után kiírja az érintett adatok értékeit a konzolra. A módosítások után a teljes DOM fát visszaírom egy új XML fájlba (XVG56E_modified.xml) egy Transformer segítségével.

```

private static void addKedvencMufajToFelhasznalo(Document doc, String
felhasznaloId, String mufajNev) {
    NodeList list = doc.getElementsByTagName("felhasznalo");
    for (int i = 0; i < list.getLength(); i++) {
        Element f = (Element) list.item(i);
        if (felhasznaloId.equals(f.getAttribute("fkod"))) {

            System.out.println("---- 2. MÓDOSÍTÁS: ÚJ KEDVENC MÚFAJ
FELHASZNÁLÓNAK ----");
            System.out.println("Felhasználó: " + felhasznaloId);
            System.out.println(" Hozzáadott kedvenc műfaj: " + mufajNev);

            // Új kedvencmufaj elem létrehozása
            Element ujMufajElem = doc.createElement("kedvencmufaj");
            ujMufajElem.setTextContent(mufajNev);

            // Hozzáadjuk a felhasznalo elemhez
            f.appendChild(ujMufajElem);

            System.out.println();
            return;
        }
    }

    System.out.println("Felhasználó nem található (fkod = " +
felhasznaloId + ")");
    System.out.println();
}

```

Az `addKedvencMufajToFelhasznalo()` metódus célja egy új, többször előforduló adat („kedvenc műfaj”) hozzáadása egy felhasználó XML eleméhez. A függvény DOM segítségével megkeresi a megfelelő `<felhasznalo>` elemet az `fkod` attribútum alapján, majd létrehoz egy új `<kedvencmufaj>` elemet, beállítja annak szövegértékét, és az `appendChild()` metódussal hozzáfűzi a felhasználóhoz. A kódrészlet jól mutatja, hogyan bővíthető egy XML dokumentum anélkül, hogy az eredeti struktúrát át kellene írni vagy újragenerálni.